

Mo1 study workbook

Questions and writing space are included. Answers and reference reasoning are withheld.

Generated 9/29/2026 · 10 sessions · Questions edition

L001 · 45 MINUTES

What system design is; diagnostic and your design notebook

drafted

LEARNING OBJECTIVES

Explain system design as a chain of justified decisions.

Distinguish state, computation, and communication in one request.

Create a notebook with assumptions, guarantees, and falsifiable evidence.

Identify a simple architecture and one honest limitation.

RETRIEVE BEFORE READING

- Recall a program you wrote that stored data. What would disappear if its process stopped?
- Name a bug where valid-looking input produced an incorrect business result.

A small problem with real consequences

Core budget: 5 minutes diagnostic, 14 minutes explanation and trace, 8 minutes notebook example, 10 minutes exercise, 5 minutes questions, 3 minutes review. No system-design vocabulary is required yet. Your Python and SQL experience is enough.

Imagine a department needs a borrowing register for 200 pieces of equipment. A student scans a tag, borrows an item, and returns it later. Start by writing what success means: an authorized student can see current availability; an item has at most one active borrower; a confirmed loan remains recorded after restarting the application. These statements immediately tell you more than a list of fashionable components.

System design connects user behavior to mechanisms and their limits. You choose where data lives, who may change it, when a response is safe to return, and how the system behaves when something fails. The design includes ordinary operation and recovery. It is a model you can challenge, not a beautiful drawing you must defend at all costs.

Before reading further, spend two minutes proposing your own design. Preserve it unchanged in the notebook. This is a diagnostic, not a graded exam: later compare how your reasoning changes. Record unknowns rather than filling them with confident guesses.

Follow one authoritative state change

Use one backend application and one persistent database. The browser expresses intent; the application checks the request; the database owns the borrowing record. That separation gives each component a responsibility. The browser must not be the authority on whether an item is available, because two browsers can have old information.

Trace a successful loan in six steps: the student submits item 42; the application identifies the student; it verifies permission; it asks the database to create a loan only if the item is free; the database persists the accepted change; the application returns the resulting loan ID. The response belongs after the persistence decision. A success response sent before saving cannot honestly mean that the loan is recorded.

Now challenge the trace. Two students can submit at the same time. Checking availability and later inserting a loan as unrelated actions allows both to pass the check. You do not need the names of transaction isolation levels yet. You do need the requirement that the check and state transition act as one protected operation. Write that requirement next to the database boundary. M04 will implement it.

A different failure happens when saving succeeds but the response is lost. The student sees an error although the state changed. A design that equates missing response with no change will create duplicate actions. For now record an unresolved outcome and a need to retrieve the current state; later lessons develop retries and idempotency. Good early design exposes such obligations rather than pretending all requests finish cleanly.

A decision notebook that supports learning

Keep four short records for each design: assumptions, decisions, unresolved questions, and evidence. An assumption might be "200 items; up to 50 simultaneously active students; one campus." A decision is "use one database as the authority for active loans." Its reason is "all competing claims on an item must be compared in one place." A rejected alternative is "browser-local storage cannot arbitrate between students." A test is "two simultaneous claims produce exactly one active loan."

Separate facts from estimates. A stakeholder-supplied item count is a fact for the exercise; a traffic multiplier you guessed is an assumption. Put a confidence level and a validation action beside uncertain numbers. Avoid inventing a precise throughput ceiling for a component you have never measured.

Here is a compact evidence entry:

Decision D1: authoritative loans live in one database.
Guarantee: at most one active loan per item.
Failure to test: process stops after saving, before response.
Evidence needed: restart and query the item; inspect active loan count.
Revisit when: one server cannot meet measured peak demand or recovery needs.

Reading about a guarantee is not evidence that you can implement it. Distinguish “I can explain,” “I have attempted,” and “I demonstrated using this test.” At this stage your evidence may be a precise trace and a counterexample; executable tests will join it later.

The first architecture is allowed to be small

One server has a shared failure domain: if it stops, the service is unavailable. Persistent storage can preserve data while the application is down, so unavailability and data loss are different problems. A backup may help recover data but does not automatically keep the service running. Conversely, two application copies cannot recover information that both depend on a lost database to read.

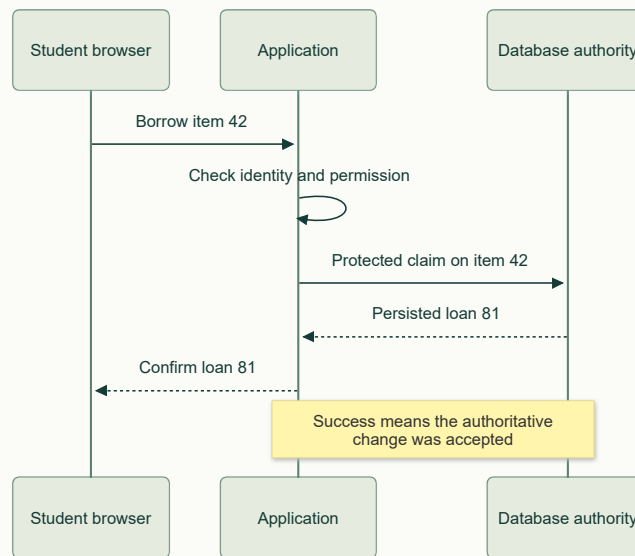
Whether that limitation is acceptable depends on the department's tolerance. If a clerk can use a temporary paper record during a short outage, a modest design may be justified. If this were emergency medical equipment, the consequences and requirements would change. The product name alone does not select the architecture.

Common mistakes are equating more boxes with more reliability, saying “scalable” without a workload, treating every failure as identical, and drawing arrows without stating whether they carry a command, a response, or stored data. Ask of every box: which requirement needs it; what state does it own; what fails if it disappears; what is the simpler alternative?

Optional practice, outside the 45-minute core: inspect one of your previous backend projects and write a missing-response failure trace. Do not add a queue or cache yet; first explain the unmet requirement.

SYSTEM VIEW

When does a loan become real?



All arrows represent a synchronous request or its response. The browser is outside the trusted application boundary. Saving precedes confirmation.

Step through the mechanism, one transition at a time.

BOUNDED EXERCISE · 10 MINUTES

In ten minutes, sketch the borrowing register and trace two students attempting to borrow the same item. Write one invariant, one outage consequence, one lost-response ambiguity, and one measurable test. Do not choose a distributed-system product.

State your assumptions. Show your reasoning.

1. Which observation is evidence of a reliable borrowing operation?

- A. The diagram has five components.
- B. The accepted loan survives a restart test.
- C. The page loads once.

2. Who should be authoritative about the active borrower?

- A. Each browser independently
- B. The persistent shared borrowing store
- C. The label printed on the item

3. A response disappears after the database saved a loan. What is known, and what remains unknown to the client?

4. When can a single-server design be a defensible decision?

MENTAL MODEL

Design is a traceable argument: requirement → state and mechanism → guarantee → failure test.

References

Google SRE: Simplicity — <https://sre.google/sre-book/simplicity/>

Amazon Jobs: SDE II Interview Prep — <https://www.amazon.jobs/content/en/how-we-hire/sde-ii-interview-prep>

Requirements, non-goals, invariants, and acceptable compromises

drafted

LEARNING OBJECTIVES

Convert vague adjectives into testable requirements.

Write non-goals that protect scope.

Express business invariants independently of implementation.

Negotiate compromises without silently weakening correctness.

RETRIEVE BEFORE READING

- From L001: where did the borrowing register keep its authoritative state?
- Explain why a missing response can leave the client uncertain.

“Build a booking system” is not yet a specification

Core budget: retrieval 5, explanation 18, worked clarification 8, exercise 10, questions 10, review 4 minutes. This lesson uses a campus rehearsal-room booking service. We deliberately separate describing required behavior from choosing transactions or locks, which come later.

A request to “make booking fast, secure, and scalable” contains aspirations but almost no acceptance criteria. Which rooms? Who is allowed to reserve them? Are overlapping reservations allowed for setup time? Can someone edit a confirmed booking? Does “fast” concern viewing availability or receiving confirmation? Clarifying these points is engineering work: each answer can change state ownership, query patterns, or the response contract.

Start with the user journey. A student browses rooms, chooses a start and end time, submits a reservation, and later cancels it. A facilities administrator can block a room for maintenance. Restrict the initial release to one campus and authenticated students. Payments, recurring bookings, and automatic conflict resolution are non-goals. A non-goal does not mean “never”; it means the current design does not promise that behavior.

Write guarantees that survive changes of technology

A functional requirement describes an action: reserve a room. An invariant constrains every valid result: two confirmed reservations for the same room must not overlap. A service objective constrains observed behavior: 99% of valid availability searches complete within 500 ms during the specified peak workload. An operational constraint limits your choices: one part-time operator and one campus deployment. Keep these categories separate because they are checked differently.

For half-open time intervals [start, end), reservations overlap when $\text{start}_A < \text{end}_B$ and $\text{start}_B < \text{end}_A$. Half-open intervals allow a booking ending at 11:00 and another beginning at 11:00, if the product permits no turnaround time. If cleaning needs ten minutes, incorporate that explicitly rather than changing comparison operators accidentally.

An invariant should describe the business, not a favored tool. "Use a lock" is not an invariant. "At most one confirmed reservation covers any instant for a room" is. Many mechanisms might enforce it. Conversely, "availability pages may lag by 30 seconds" can be an acceptable compromise only if the final confirmation still arbitrates the authoritative state. A stale hint is not permission to violate the booking rule.

Scope each guarantee. "No duplicate bookings" could mean no overlapping room occupancy or no repeated result from one retried action. Those are different conditions. Write the exact entities and transitions. Ask what happens if two students reserve simultaneously or an administrator closes a room while a student is choosing it.

A clarification exchange and its consequences

Candidate: "Do we need global rooms or one campus?" Stakeholder: "One campus, 40 rooms." Consequence: no global placement requirement has been established.

Candidate: "Is a displayed slot guaranteed until the user submits?" Stakeholder: "No, but an accepted reservation must not conflict." Consequence: searching can provide a provisional view while confirmation needs an authoritative check.

Candidate: "Can the system reject new bookings if it cannot verify availability?" Stakeholder: "Yes; browsing can stay available from a slightly old view." Consequence: different operations have different failure policies. Showing old availability may be tolerable; inventing a confirmation is not.

Candidate: "What is most painful: a slow page or two groups arriving for the same room?" Stakeholder: "Double booking is worse." Consequence: when speed and correctness conflict, preserve the invariant and return a clear unsuccessful or unresolved result.

Record the result as a contract:

```
Action: confirm a reservation for room_id, start, end, student_id.  
Preconditions: start < end; student authorized; room usable.  
Accepted result: reservation ID with a confirmed time interval.  
Invariant: confirmed intervals for the same room do not overlap.
```

Rejected result: explicit conflict, invalid input, or unavailable decision.
Non-goals: payments, recurring series, cross-campus operation.

This does not yet implement safe concurrency. It tells later mechanisms what they must prove. If you cannot describe a counterexample, your requirement may be too vague to test.

Negotiation is explicit, not accidental

Some requirements can move: result freshness, image quality, or the size of a search page. Others may define the product's validity: isolation between private accounts or preserving confirmed reservations. You may negotiate either category with the relevant owner, but must not silently trade away an invariant because a component is inconvenient.

Consider three alternatives during an outage. Reject confirmation and keep browsing: bookings are temporarily unavailable, but no unverified booking is accepted. Accept a tentative request and clearly label it pending: this changes the API and user expectation; it is not a confirmed booking. Accept every request as confirmed and reconcile later: this breaks the stated invariant and requires a different product agreement. Naming a queue does not make the third option correct.

Failure scenarios should be requirement probes. If deletion of a student account must remove their private data, what happens to an audit record? The answer may require separate retention policies and pseudonymous references, rather than a blanket deletion claim. Do not invent legal deadlines; record the policy owner and a question.

Common misconceptions: non-functional means optional; low latency means low average only; a non-goal is irrelevant forever; choosing a database proves business correctness. Each mistakes a label for an observable contract. End clarification by reading back a small list of agreed guarantees and the most consequential uncertainty.

SYSTEM VIEW

An available-looking slot is only a proposal

The diagram distinguishes a convenient view from a correctness decision. "Pending" must be a documented product state, never a disguised confirmation.

Step through the mechanism, one transition at a time.

BOUNDED EXERCISE · 10 MINUTES

A stakeholder says "Build a fast parcel-locker booking app." In ten minutes write three clarifying questions, two non-goals, two invariants, one measurable service objective, and an outage compromise. Include what would

make your compromise unacceptable.

State your assumptions. Show your reasoning.

1. Which is a business invariant?

- A. Use PostgreSQL.
 - B. At most one active allocation per locker.
 - C. The service should be modern.
-
-
-

2. A stale availability page is permitted. What follows?

- A. Confirmation may ignore conflicts.
 - B. Confirmation still must preserve the agreed invariant.
 - C. All operations must use stale data.
-
-
-

3. Which clarification is most useful first?

- A. Which queue brand should we use?
 - B. What outcome would be unacceptable even during failure?
 - C. Should the logo be blue?
-
-
-

4. Explain the difference between pending and confirmed during an outage.

5. Should a faster confirmation path be adopted if it can occasionally double-book?

MENTAL MODEL

Clarify what may be approximate, what must remain true, and what the system is allowed to say when it cannot decide.

References

Google SRE: Service Level Objectives — <https://sre.google/sre-book/service-level-objectives/>

Latency, throughput, availability, durability, and service objectives

drafted

LEARNING OBJECTIVES

- Distinguish latency, throughput, availability, and durability.
- Interpret percentiles without confusing them with maximum latency.
- Define a user-facing indicator and its target.
- Compute a simple request-based error budget.

RETRIEVE BEFORE READING

- From L002, give a functional requirement and a separate invariant.
- What does a confirmed response promise in the borrowing example?

Four questions, four different measurements

Core budget: 5 minutes retrieval, 18 explanation, 8 arithmetic, 10 exercise, 10 questions, 4 review. Reuse a photo-notebook service: a user saves a photo with a caption, then opens an album. This is an invented workload, not a description of a commercial product.

Latency asks how long one operation takes. Throughput asks how many operations cross a specified boundary per unit time. Availability asks whether the requested useful service can be obtained. Durability asks whether accepted information remains preserved over the stated period and failures. A service may be available but return the wrong account's photos, so correctness is not supplied by any of these performance numbers.

An upload can take two seconds while the system finishes 500 uploads per second if many uploads overlap. There is no contradiction: one is a duration and one is a rate. A process can be down while its stored photos remain safe. Conversely, it can answer every request promptly after accidentally deleting half the albums. That is high request availability for some operations and terrible data preservation.

Always name the operation and observation point. Upload-start to first response, upload-start to fully stored photo, and album-click to visible image are different latencies. A server-side timer cannot see all time spent waiting in the browser or traversing the network.

The average can hide the user who is stuck

Suppose 100 album requests contain 95 durations of 100 ms and five durations of 2,000 ms. Their arithmetic mean is $(95 \times 100 + 5 \times 2,000) / 100 = 195$ ms. Using nearest-rank percentiles, p95 is 100 ms and p99 is 2,000 ms. A p95 target alone permits the slowest five percent to be much worse. p99 is not the maximum, and a percentile from 100 observations is much less stable than the same percentile from a large representative population.

Specify how the distribution is gathered. Mixing image thumbnails with huge uploads can make a single latency number unhelpful. Counting only successful responses can conceal fast failures or exclude the worst timeouts. Measure errors and success latency together, and explain your treatment of timed-out operations.

Do not average the p99 values of two servers to obtain a fleet p99. One server may have ten requests and the other ten thousand; even weighting their percentile values does not reconstruct their distributions. Combine the underlying observations or compatible histograms. The purpose of the percentile is to describe a population, not to provide a convenient number detached from that population.

In the walkthrough, predict where the user-visible timer starts and stops. If a backend reports 80 ms but an image becomes visible after 900 ms, investigate the gap before claiming the database is slow.

Turn a measurement into a decision

An SLI is the measurement definition; an SLO is a target for that measurement. A practical album-read SLI might be the fraction of valid album reads that return the authorized album within 500 ms, measured at the local gateway during a rolling 28-day window. An initial SLO might require that fraction to be at least 99.5%. Those numbers are exercise assumptions and require product agreement in real work. An SLA adds an agreement and consequences; it is not merely another spelling of SLO.

For 200,000 eligible requests, a 99.5% success target permits 1,000 bad requests in the window: $200,000 \times (1 - 0.995)$. If 700 are already bad, 300 remain under this simplified fixed-volume calculation. A moving window and future request volume change the available budget, so a production dashboard needs exact window semantics.

Do not convert that request budget into minutes of downtime without assumptions. A five-minute outage during a busy event may affect more users than an hour overnight. A time-based objective is a different measure. Likewise, a storage durability target needs a definition of accepted data and failure scope; “we have backups” is a mechanism claim, not a measured loss probability.

An objective earns its place when missing it leads to an action: reduce overload, investigate a slow dependency, or improve recovery. CPU utilization is often diagnostic evidence. It does not by itself tell you whether users can complete the operation.

Use the dimensions to diagnose, not decorate

Consider a deployment that doubles completed requests per second but increases p99 from 400 ms to 4 seconds. It improved one rate while potentially violating the interactive objective. A batch import might welcome that tradeoff; a user waiting for a booking decision may not. Decide using the operation's needs.

Now stop the application process while leaving its disk intact. Availability drops; no data loss has yet been shown. Restore the process from an old snapshot that misses yesterday's confirmed photos. Availability returns, but durability has failed for those photos. Your report should describe both events separately so the team does not celebrate recovery while overlooking missing data.

Common mistakes include claiming "99.9% uptime" without a window, promising "zero latency," excluding all failures from the denominator, interpreting p99 as a guarantee on every request, and treating replication or backup count as proof of durability. Prefer a small set of precise measurements to a page of unexplained percentages.

Optional depth: instrument a familiar script with a monotonic timer and collect a distribution. This extension is separate from the scheduled arithmetic exercise; no instrumentation library is required for the core.

SYSTEM VIEW

Which latency does the user experience?

Server latency is a subset of the journey. The drawing shows ordering, not measured durations.

Step through the mechanism, one transition at a time.

BOUNDED EXERCISE · 10 MINUTES

An API receives 80,000 eligible requests. 240 fail, and another 160 return correctly but slower than the 600 ms objective. Define a combined good-request SLI, compute its value, and decide whether it meets 99.6%. Explain what a backup does and does not tell you about this result.

State your assumptions. Show your reasoning.

1. Which has units of requests/second?

- A. Latency
- B. Throughput
- C. Durability

2. p99 is 800 ms. What does this describe?

- A. Every request is below 800 ms.
- B. The 99th-percentile point of the defined observed population.
- C. The mean is 800 ms.

3. At a 99.9% target, how many bad requests are allowed among 500,000 eligible requests?

4. The service is available after restoration but loses the last hour of confirmed writes. Which claims must be separated?

5. When might you accept a throughput gain with worse latency?

MENTAL MODEL

Duration, rate, usability, and preservation are separate promises. Define the population before interpreting a number.

References

Google SRE: Service Level Objectives — <https://sre.google/sre-book/service-level-objectives/>

Google SRE: Monitoring Distributed Systems — <https://sre.google/sre-book/monitoring-distributed-systems/>

Estimation I: users, actions, average traffic, and peak traffic

drafted

LEARNING OBJECTIVES

Translate active users and actions into operation rates.

Separate average traffic from a stated peak scenario.

Account for API amplification and workload mix.

Use ranges and dimensional checks instead of false precision.

RETRIEVE BEFORE READING

- From L003: explain why requests/second and milliseconds cannot substitute for each other.
- From L002: name a non-goal before estimating a product.

Count behavior before servers

Core budget: retrieval 5, explanation 18, worked arithmetic 8, exercise 10, questions 10, review 4 minutes. We estimate a reading-list app where users save links and browse a personal list. The figures are explicit invented assumptions, not measurements from an existing service.

Registered accounts are not a request rate. Start with a population active within a stated interval, multiply by actions per active person in that interval, and divide by interval seconds. Units should cancel: $\text{users/day} \times \text{actions/user} \times \text{requests/action}$ gives requests/day . Dividing by seconds/day yields requests/second . In everyday interview shorthand “DAU $\times$ actions per user per day” means the same calculation, but writing units prevents accidentally multiplying by an extra day.

Suppose the app has one million registered accounts, 100,000 daily active users, and each active user opens their list 12 times and saves two links per day. Browse volume is 1.2 million operations/day; saves are 200,000/day. Average browse rate is about 13.9/s, save rate 2.31/s, and combined rate 16.2/s. One million accounts alone could have suggested an enormous service while hiding a modest actual flow.

An action is not necessarily an API request. If opening a list fetches a page of data and then separately fetches permissions and unread counts, one action makes three API calls. If each page triggers one query per item, internal database demand may be amplified again. Keep user actions, external requests, and internal operations in separate rows.

A peak needs a timescale and a cause

Daily average rate spreads work across all 86,400 seconds. If most users are in one time zone or a notification prompts them simultaneously, that average misses the decisive load. For this exercise, assume a busiest one-minute window at eight times the daily mean. External peak is then about 130 requests/s under the one-call-per-action model. Eight is a scenario assumption to validate, not a universal industry multiplier.

A one-second burst, one-minute peak, and one-hour peak can require different handling. A buffer may absorb a brief surge for delay-tolerant work; it cannot erase sustained demand that exceeds completion capacity. An interactive deadline can also make buffering unsuitable. State whether arrivals are spread evenly within your peak window; 7,800 requests in one minute could arrive smoothly or in one second.

Traffic mix may change during the peak. During an import launch, saves might dominate, even though reads dominate the daily average. A single overall multiplier assumes the mix stays fixed. Split the estimate when the architecture is sensitive to write cost, expensive search, large uploads, or a hot account.

Geographic and tenant skew also matter. A global average does not say how much reaches one region or one customer's partition. Record the largest plausible concentration before assuming traffic is evenly spread.

An estimate you can recompute aloud

Use round numbers until a decision needs precision. Here is an executable standard-library arithmetic example, intended for Python 3.11 or newer. Save as `traffic.py` and run `python traffic.py`. Expected output is shown; the companion verification script checks these equations.

```
def average_rps(dau: int, actions_each: int, calls_each: int = 1) → float:
    assert dau ≥ 0 and actions_each ≥ 0 and calls_each ≥ 0
    return dau * actions_each * calls_each / 86_400

browse = average_rps(100_000, 12)
save = average_rps(100_000, 2)
assert round(browse + save, 1) == 16.2
print(f"average={browse + save:.1f}/s peak={(browse + save) * 8:.1f}/s")
# average=16.2/s peak=129.6/s
```

Sensitivity is more useful than extra decimal places. If daily active users are between 50,000 and 200,000, the same behavior produces 8.1–32.4 requests/s average and roughly 65–259/s at eightfold peak. If browse requires three calls, combined daily calls become 3.8 million, or 44.0/s average and 352/s peak. That uncertainty changes the result more than rounding 86,400 to 100,000.

Do not conclude “one server supports this” without measurements and request cost. State instead: “The demand model is about 130/s peak under these assumptions; next I need per-operation service cost and a representative benchmark.”

Use the number to choose the next question

The estimate should answer a design question. If the initial concern is network capacity, request counts alone are insufficient; the next lesson introduces bytes. If the concern is memory held by unfinished requests, duration matters; L007 connects rate and concurrency. If the concern is a business correctness race, even two simultaneous writes can violate an invariant, so low traffic is no excuse to ignore it.

Failure scenarios begin in the workload model. A client retry after every timeout can add attempted requests without increasing useful user actions. A periodic job scheduled for every tenant at midnight can produce a concentrated burst. A product announcement can synchronize readers on one object. Record these as scenarios rather than quietly folding them into a vague safety factor.

Common errors include using monthly active users as daily active users, dividing by hours while labeling the result per second, applying a peak multiplier twice, estimating every operation as equally expensive, and confusing completed throughput with offered demand during overload. Use a small table with population, action, interval, multiplier, and uncertainty to expose these errors.

Optional practice: estimate one day of a service you use, with an honest low/high range and a plan to measure it. Do not research private traffic numbers and present guesses as company facts.

SYSTEM VIEW

Turn human actions into component demand

This is a demand calculation, not a measured benchmark or server-capacity claim. The one-call assumption is deliberately visible.

Step through the mechanism, one transition at a time.

BOUNDED EXERCISE · 10 MINUTES

Estimate a course app with 60,000 daily active students. Each opens 20 lesson pages and submits three quizzes daily. Each page makes two API calls; each quiz submission makes one. Assume a sixfold peak with the same mix. Compute average and peak API requests/s, then name two assumptions that could invalidate this peak estimate.

State your assumptions. Show your reasoning.

1. 10,000 users each make 86.4 requests/day. What is average demand?

2. A sixfold peak multiplier is best described as:

- A. A universal constant
- B. A stated workload assumption requiring validation
- C. A guarantee provided by a load balancer

3. Which can increase requests without increasing useful user actions?

- A. Retries after timeouts
- B. Renaming an endpoint
- C. Reducing registered-account count only

4. Why is a per-operation traffic mix useful?

5. Is one server enough for 200 requests/s?

MENTAL MODEL

Population × behavior × amplification ÷ time. Every multiplier is an assumption you should be able to explain.

References

Google SRE: Monitoring Distributed Systems — <https://sre.google/sre-book/monitoring-distributed-systems/>

[A] Clarify a vague product request and defend its estimates

drafted

LEARNING OBJECTIVES

Clarify the first-release scope without component shopping.

Calculate a defensible request model and challenge its assumptions.

Explain private-state and accepted-write guarantees.

RETRIEVE BEFORE READING

Assessment session

This catalog slot uses interview_001, a concept-check assessment. Start it in the Interview Center. The scheduled session is 60 minutes: 45 minutes of answering followed by 15 minutes of evidence-based feedback. The candidate brief is revealed when the attempt begins. Coaching mode permits requested hints; exam mode records hints and keeps the reference solution for the debrief. Do not use a normal lesson quiz as a substitute for this session.

BOUNDED EXERCISE · 45 MINUTES

Start the mapped interview, save your actual reasoning, then compare it with the rubric after submission.

State your assumptions. Show your reasoning.

MENTAL MODEL

Demonstration requires an attempted explanation and recorded evidence; opening this page awards no proficiency.

References

Amazon Jobs: SDE II Interview Prep — <https://www.amazon.jobs/content/en/how-we-hire/sde-ii-interview-prep>

Estimation II: bytes, retention, storage, and bandwidth

drafted

LEARNING OBJECTIVES

Derive retained logical data from writes, record size, and retention.

Separate logical size from replicas, indexes, backups, and free space.

Convert bytes/s to bits/s correctly.

Identify what dominates a storage or bandwidth estimate.

RETRIEVE BEFORE READING

- From L004: why does registered-user count not determine traffic?
- From L003: distinguish durability from availability.

A request rate does not tell you how much data exists

Core budget: retrieval 5, explanation 18, worked calculation 8, exercise 10, questions 10, review 4 minutes.

Consider a field-observation app where students attach a photo to a short record. Use decimal units throughout: 1 kB = 1,000 bytes, 1 MB = 1,000,000 bytes, and 1 GB = 1,000,000,000 bytes. Binary KiB/MiB/GiB are legitimate but different; always state the convention.

Assume 200,000 new observations daily. Each contains 1,500 bytes of metadata and one compressed 2 MB photo. Retain both for 90 days. Logical metadata growth is 300 MB/day; photos grow by 400 GB/day. Before adding any infrastructure multiplier, the photo path is already more than a thousand times larger. Optimizing metadata size by 20% barely changes the overall storage decision.

The basic retained-data model is $\text{writes/day} \times \text{bytes/write} \times \text{retained days}$, assuming steady arrivals, fixed record sizes, and timely deletion. Updates complicate it: overwriting a record may not grow live logical data, while version history and logs do. Deletion removes live records according to policy but may not immediately reclaim physical space or remove copies from backups. The model must say which population it counts.

Build a ledger of copies and overheads

Metadata live size is $200,000 \times 1,500 \times 90 = 27$ GB. Photo live size is $200,000 \times 2,000,000 \times 90 = 36$ TB. Suppose the teaching design keeps three full metadata copies, one additional index allocation equal to 40% of base metadata on each copy, and 30% free-space reserve expressed as usable data occupying 70% of provisioned disk. Metadata provisioned capacity is $27 \times 1.4 \times 3 / 0.7 = 162$ GB.

Notice the reserve convention. "Add 30%" produces 147.42 GB, where the reserve is 30% of used bytes but only about 23% of provisioned capacity. "Leave 30% of the disk free" means divide by 0.7. Both can be legitimate planning conventions; using the same phrase for both causes mistakes.

Do not apply the metadata replication factor blindly to photos. Their chosen storage system may use replication or erasure coding with a different physical multiplier. For this exercise use two full photo copies: 72 TB before free space and backups. This is an arithmetic assumption, not a recommendation for a durability target. Copy count alone says nothing about independent failure domains or accidental deletion.

Keep backups in another ledger. A daily full backup retained for seven days is very different from incremental changed blocks with periodic full snapshots. State the mechanism before multiplying. Temporary migration copies, compaction space, and restore workspaces can dominate a nearly full system; these belong in peak physical capacity, not just steady live data.

Follow the bytes in each direction

Suppose observations arrive at a peak of 40 writes/s. Metadata ingress is 60,000 bytes/s. Photo ingress is 80 MB/s, which is 640 megabits/s before protocol overhead: multiply bytes by eight. A nominal 1 gigabit/s link does not promise that an application will sustain the full nominal rate; framing, encryption, competing traffic, and actual network paths need measurement.

Now consider reads. If 100 users/s each view a page with 12 thumbnails of 40 kB each, thumbnail egress is 48 MB/s or 384 megabits/s. If the page accidentally downloads twelve full 2 MB photos instead, egress becomes 2.4 GB/s or 19.2 gigabits/s. Representation size changed demand by fifty times without changing user count or page RPS.

Replication traffic is a different flow. Each new 2 MB photo copied once creates another 2 MB internal transfer. Restoring 10 TB over an effective 100 MB/s path takes 100,000 seconds, roughly 27.8 hours, before validation and operational overhead. Thus a bandwidth estimate can invalidate a recovery promise even when normal requests fit.

An arithmetic-only Python example:

```
def bits_per_second(rps: float, payload_bytes: int) -> float:
    return rps * payload_bytes * 8

assert bits_per_second(40, 2_000_000) == 640_000_000
restore_hours = 10_000_000_000_000 / 100_000_000 / 3600
```

```
assert round(restore_hours, 1) == 27.8
print("photo ingress: 640 Mbit/s; minimum restore: 27.8 h")
```

Run with Python 3.11+ using only the standard library. These are dimensional calculations, not a network benchmark.

Question the largest and least certain terms

Large payloads often have skewed size distributions. A mean is useful for total stored volume, but a maximum or high percentile may matter for a single-request memory limit. Compression depends on actual data; a tenfold assumed compression ratio is not free capacity until tested on representative samples. Encrypted or already compressed content may compress poorly.

A retention backlog changes live storage. If cleanup stops for five days, this workload adds another 2 TB of photos before copies. A logging mistake that stores entire payloads can create an unexpected second data lake. A backup job can saturate the same link used for foreground requests even though the average daily traffic appears modest.

Common misconceptions: all data gets the same replication multiplier; replicas replace backups; storage cost is only live rows; bits and bytes are interchangeable; user uploads equal total network traffic. Trace every major copy and transfer explicitly. End with the dominant quantity and the question that most reduces uncertainty.

Optional practice outside the core: measure the encoded sizes of ten real sample records, without uploading private data. Compare average, maximum, and a justified planning value.

SYSTEM VIEW

Where does one observation become several allocations?

Solid arrows are upload flows; dotted arrows create additional copies. The simplified picture does not imply a production durability guarantee or exact physical layout.

Step through the mechanism, one transition at a time.

BOUNDED EXERCISE · 10 MINUTES

A sensor service stores 5 million readings/day at 200 bytes each for 30 days. Indexes add 50% to base data. Keep two full copies and leave 25% of allocated disk free. Compute live base data, provisioned storage excluding backups, and peak ingress at 20,000 readings/s. Use decimal units.

State your assumptions. Show your reasoning.

1. 2 million 500-byte records each day for 10 days: logical storage in decimal GB?

2. Convert 12 MB/s to megabits/s using decimal units.

3. If 90 GB used must occupy at most 75% of disk, what allocation is needed?

- A. 112.5 GB
 - B. 120 GB
 - C. 90 GB
-
-
-

4. Why are replicas and backups separate budget items?

5. Which should you investigate first: a 20% uncertainty in 27 GB metadata or a 2× uncertainty in 36 TB photos?

MENTAL MODEL

Count logical information once, then account for every copy, representation, retention rule, and byte transfer.

References

Google SRE: Service Level Objectives — <https://sre.google/sre-book/service-level-objectives/>

Estimation III: concurrency, Little's law, and queueing intuition

drafted

LEARNING OBJECTIVES

- Use Little's law with a consistent boundary and averages.
- Distinguish arrival rate, concurrency, and worker count.
- Explain why a queue grows when arrivals exceed completion capacity.
- Estimate backlog drain time and challenge stability assumptions.

RETRIEVE BEFORE READING

- From L004: what is the difference between offered demand and completed throughput?
- From L003: why is p99 not an average?

Requests occupy the system while time passes

Core budget: retrieval 5, mechanism 18, worked model 8, exercise 10, questions 10, review 4 minutes. Picture a service as a room with an entrance and an exit. Arrival rate counts how quickly work enters; concurrency counts how much unfinished work is inside; latency measures how long each item remains. Worker count tells you how many processing slots exist, which is related but not identical to unfinished work.

For a stable long-run system, Little's law is $L = \lambda W$: average items in the chosen system equals average effective throughput times average time spent there. If 200 requests/s complete and mean end-to-end time is 0.15 s, average in-flight requests are 30. The units cancel: requests/s $\times$ s = requests. The result says nothing by itself about CPU utilization or the number of operating-system threads.

Choose the boundary before using the equation. If time includes waiting and execution, L includes queued and executing requests. If time covers only a database call, L counts database calls inside that boundary. Mixing total application latency with database-only throughput produces a number with no coherent interpretation.

Use compatible averages over a representative stable period. Plugging p99 latency into Little's law does not yield p99 concurrency. Correlations and distributions matter for tails. A continuously growing queue is not in the steady state assumed by the simple long-run calculation; it needs a transient backlog model.

The queue records work you have not finished

Use a deliberately simple fluid model: in each one-second interval, 120 jobs arrive and workers can finish 100. Starting empty, backlog grows by 20 jobs/s: after ten seconds, 200 jobs wait. The model smooths event timing and job sizes; it is an illustration, not a prediction of real percentile latency.

When arrivals fall to 80/s, spare completion capacity is 20/s, so those 200 extra jobs take ten seconds to drain. Dividing by total capacity would give two seconds and ignore the ongoing new arrivals. If arrivals stay at 100/s, there is no spare rate to clear the backlog. If each job times out before reaching a worker, the system must discard or cancel it deliberately; otherwise workers may spend capacity on results nobody needs.

A queue does not add service capacity. It can absorb a finite burst by exchanging memory or durable storage and waiting time for smoother work. For an interactive operation with a short deadline, that exchange may be unacceptable. A finite queue plus explicit admission policy can provide a predictable failure instead of allowing memory to grow until the process dies. The detailed policies arrive in M10.

Even average demand below capacity does not eliminate waiting. Real arrivals bunch together and job durations vary. A worker may be busy with a slow item when several short ones arrive. As spare capacity shrinks, there are fewer opportunities to catch up after variation. The exact delay distribution requires a more specific model or measurement; no universal "safe CPU percentage" follows from this lesson.

Concurrency is a resource budget

Suppose a report endpoint accepts 50 requests/s and each request spends an average 2 seconds waiting for a remote source plus 0.1 seconds computing. Average end-to-end concurrency is approximately $50 \times 2.1 = 105$ requests, provided the system is stable and the measurements cover the same population. If each in-flight request retains a 4 MB buffer, those buffers alone average roughly 420 MB.

The endpoint is not necessarily using 105 CPU cores. Waiting may consume a socket and memory while little CPU executes. Conversely, CPU-heavy work needs actual execution capacity. Distinguish the resource occupied during each phase before choosing a concurrency model. M03 connects this to threads and event loops.

A small deterministic Python simulation of the earlier queue:

```
def step(backlog: int, arrivals: int, capacity: int) → int:
    return max(0, backlog + arrivals - capacity)

backlog = 0
for _ in range(10):
    backlog = step(backlog, 120, 100)
assert backlog == 200
for _ in range(10):
    backlog = step(backlog, 80, 100)
assert backlog == 0
print("peak backlog=200; drained after 10 seconds")
```

This executable Python 3.11+ standard-library model treats arrivals and completion capacity as aggregate one-second quantities. It does not simulate individual service times, ordering, retries, or a real scheduler. Its test establishes the arithmetic behavior of the model only.

Slow dependencies become local pressure

At unchanged arrival rate, a slower downstream service makes requests remain in your process longer. A jump from 100 ms to 2 s at 100 requests/s changes average in-flight work from 10 to 200. Memory, connections, and file descriptors can fill before CPU looks busy. Retrying adds arrivals, potentially making the original slowdown worse.

A concurrency limit creates another boundary. If you allow only 50 downstream calls at once and each holds its slot for 2 s, a rough saturated completion bound is 25 calls/s under those assumptions. Accepting 100/s without another policy accumulates waiting work. Raising the limit might improve throughput if the dependency has spare capacity, or overload it further if it is already saturated. Measurement and dependency limits decide.

Common misconceptions: throughput equals concurrent users; average latency can be replaced with p99 in the equation; a queue solves overload forever; a larger thread pool creates CPU; low average utilization proves no burst risk. Keep the boundaries and assumptions visible.

Optional depth: instrument a workload to estimate average in-flight work from area under a concurrency-versus-time curve, then compare it with completion rate $\times$ mean duration. Account for work already present at the window start and unfinished at its end.

SYSTEM VIEW

What is inside Little's-law boundary?

L counts both waiting and executing jobs when W covers their full time inside this boundary. The dotted annotation is not a data flow.

Step through the mechanism, one transition at a time.

BOUNDED EXERCISE · 10 MINUTES

A stable endpoint completes 80 requests/s at mean duration 250 ms. Estimate average in-flight requests. Then a burst delivers 140 jobs/s to a 100 jobs/s worker system for five seconds. How large is the extra backlog, and how long to drain it when arrivals drop to 75/s?

State your assumptions. Show your reasoning.

1. Stable throughput 300/s and mean time 0.2 s imply what average concurrency?

2. Can p99 time replace mean time in Little's law?

- A. Yes, yielding p99 concurrency
 - B. No, the simple relation uses compatible averages
 - C. Only when measured in milliseconds
-
-
-

3. Backlog 600, capacity 100/s, ongoing arrivals 80/s: fluid-model drain time?

4. Why might memory rise while CPU stays low after a dependency slows?

5. When can buffering a burst be a valid compromise?

MENTAL MODEL

A slowdown keeps work inside longer; the extra work must fit somewhere or be rejected.

References

Google SRE: Handling Overload — <https://sre.google/sre-book/handling-overload/>

Estimation IV: headroom, bottlenecks, sensitivity, and cost

drafted

LEARNING OBJECTIVES

- Find the limiting resource from a per-operation demand model.
- Calculate headroom with explicit utilization and failure assumptions.
- Perform sensitivity analysis on uncertain inputs.
- Compare operating cost without treating illustrative prices as current quotes.

RETRIEVE BEFORE READING

- From L007: how long does a backlog take to drain when new work still arrives?
- From L006: distinguish live data and provisioned capacity.

A pipeline is limited by its tightest relevant resource

Core budget: retrieval 5, explanation 18, worked decision 8, exercise 10, questions 10, review 4 minutes. A campus image converter accepts a picture and returns a thumbnail. Assume each request consumes 20 ms of CPU, reads 1 MB, writes 100 kB, and has a 500 ms interactive deadline. These costs are supplied teaching measurements; they do not describe your computer or a cloud instance.

An eight-core machine has at most eight CPU-seconds available per wall-clock second. At 0.020 CPU-seconds/request, the ideal CPU bound is 400 requests/s. This is a resource-accounting upper bound, not a safe benchmark result. It ignores scheduling overhead, other processes, uneven task sizes, and contention. If you target at most 60% CPU use under the representative peak, the planning CPU capacity becomes 240 requests/s.

Suppose effective storage read capacity is 150 MB/s. At 1 MB/read per request it bounds demand at 150 requests/s, below the CPU planning value. More CPU alone cannot fix that read bottleneck. Reducing read bytes, improving the storage path, or exploiting valid reuse might. The requirement and access pattern choose the remedy.

Some resources are counts rather than rates: maximum open connections, memory for concurrent work, or available identifiers. Check them using the right model. A service may fit average disk bandwidth but exhaust memory during a slow dependency, as L007 showed.

State exactly which bad day you are reserving for

Headroom is spare capacity relative to an explicit scenario. It may cover workload uncertainty, burstiness, recovery, or a failed instance. If one machine safely sustains 100 requests/s at the required latency and peak demand is 180/s, two machines provide only 20/s spare while both are healthy. Losing one leaves half the fleet and too little capacity. Three such machines provide 200/s with one lost, assuming balanced routing and no shared bottleneck.

For identical independent instances, a simple one-failure sizing rule is $n \geq \text{ceil}(\text{peak} / \text{sustainable_capacity_per_instance}) + 1$. It only applies if the measured per-instance capacity remains valid together, failover works, data dependencies still fit, and the failure does not remove more than one instance. Two copies on the same host do not satisfy a host-failure assumption.

Avoid hiding every uncertainty in one multiplier. A peak factor describes demand shape. A target utilization describes operating margin. A failure reserve describes lost capacity. They can all be appropriate, but each should have a purpose so you can detect double counting. Do not multiply peak demand by two “for safety” and then also require 50% utilization without explaining whether those cover distinct risks.

Spare capacity is not instantaneous recovery. Detection, rerouting, warm-up, and data loading take time. A design can have enough theoretical surviving capacity while violating latency during the transition.

A small table can change the architecture

For the image converter, evaluate three input-size scenarios:

Average source bytes	Read bound at 150 MB/s	Consequence
0.5 MB	300 requests/s	CPU planning bound of 240/s dominates
1 MB	150 requests/s	Storage reads dominate
3 MB	50 requests/s	Storage dominates much earlier

This table identifies a useful next measurement: source-image size and effective read throughput. It also prevents an expensive response to the wrong bottleneck. Buying more CPU while images triple in size does not follow from the model.

Now compare two hypothetical plans. Plan A costs 100 cost-units/month per machine and needs three machines. Plan B costs 180 per machine, sustains twice the tested rate, and needs two machines for the same one-failure scenario. Machine cost is 300 versus 360 units. But Plan B may reduce patching or coordination work; Plan A may offer a smaller failure increment. These are invented cost units, not cloud price quotes. Add storage, network transfer, backups, and operator effort before choosing.

Arithmetic implementation, executable with Python 3.11+:

```

from math import ceil

def instances(peak_rps: float, safe_rps: float, failed: int = 1) → int:
    assert peak_rps ≥ 0 and safe_rps > 0 and failed ≥ 0
    return ceil(peak_rps / safe_rps) + failed

assert instances(180, 100) == 3
assert instances(180, 200) == 2
print("A: 3 instances; B: 2 instances")

```

The function validates a sizing formula, not routing behavior or real-world service capacity.

Scaling evidence must include the shared dependency

Suppose each application copy individually reaches 100/s, but all copies share a database capped at 150 useful operations/s. Three copies do not produce 300/s. Parallelism increases pressure on the same authority. Benchmarking one copy then multiplying by fleet size is a hypothesis to test, not demonstrated scalability.

A failure can also change request cost. If cached thumbnails disappear, each request may read and convert the original image. A replica recovering from disk can compete with foreground reads. Capacity planning should therefore name degraded modes, not only the happy-path peak.

Common misconceptions include “70% CPU is always safe,” “autoscaling has no delay,” “a larger machine fixes every bottleneck,” “cost equals the server bill,” and “twice the components means twice the capacity.” The correction is not another universal percentage; it is a demand-per-resource model with measured sustainable limits and failure assumptions.

Finish a capacity argument in three sentences: identify the limiting resource under the current model; state the amount of margin and the event it covers; name the measurement or requirement change that would make you revisit the design. Optional depth is a real benchmark on your machine, explicitly separate from the supplied illustrative measurements.

SYSTEM VIEW

Which resource limits the current request?

The calculation compares supplied resource budgets. The result remains an estimate until an integrated workload test confirms it.

Step through the mechanism, one transition at a time.

A service peaks at 260 requests/s. A representative benchmark shows one instance sustaining 100/s at the required latency, but its shared database sustains only 220/s. Explain why adding a fourth application instance alone does not meet the requirement. Give a one-instance-loss sizing calculation after the database is improved to 500/s. Identify one remaining failure assumption.

State your assumptions. Show your reasoning.

1. 4 CPU cores, 10 ms CPU/request: ideal CPU-only upper bound?

2. A shared dependency caps useful throughput at 200/s. Adding app replicas necessarily:

- A. Increases throughput beyond 200/s
- B. May increase pressure without lifting the shared limit
- C. Eliminates queueing

3. Peak 350/s, sustainable instance rate 100/s, reserve for one lost instance. How many instances?

4. Why state the purpose of each headroom factor?

5. When would the cheaper server plan be more expensive overall?

MENTAL MODEL

Demand consumes several budgets; the smallest remaining budget limits you, and the reserve must cover a named event.

References

Google SRE: Handling Overload — <https://sre.google/sre-book/handling-overload/>

The 45/60-minute interview framework; diagrams and decision records

drafted

LEARNING OBJECTIVES

Allocate interview time while preserving a complete reasoning chain.

Choose architecture, sequence, and failure views for different questions.

Write a concise decision record with alternatives and revisit triggers.

Adapt a design when an assumption changes.

RETRIEVE BEFORE READING

- From L002: which guarantee must survive a convenience tradeoff?
- From L008: explain the difference between peak demand and failure reserve.

Time is another constrained resource

Core budget: retrieval 5, framework and diagrams 18, worked rehearsal 8, exercise 10, questions 10, review 4 minutes. This session teaches the curriculum's interview structure. It is a training method, not a claim that every employer uses the same schedule. Official interview guidance can set broad expectations, but the allocations here are original to your supplied curriculum.

For 45 minutes, allocate 5 to scope and invariants, 4 to useful workload estimates, 6 to APIs and data ownership, 7 to an initial architecture and request trace, 12 to the most consequential deep dives, 7 to failures and operational constraints, and 4 to synthesis and changed requirements. The numbers sum to 45. For a 60-minute interview the corresponding allocations are 7, 5, 8, 10, 16, 9, and 5.

Treat these as checkpoints, not a waterfall. Security shapes the API and data model early. Failure reasoning may expose a missing invariant during clarification. If the interviewer introduces a new requirement, revisit the relevant decision instead of saving all adaptation for the last minute. In a normal course mock, 45 minutes of interviewing and 15 of feedback fit a 60-minute session; L297 uses a separate 60-minute interview followed by its next-session debrief.

The most common timing failure is spending twenty minutes enumerating features or estimating bytes that never affect the design, then rushing through an unexplained diagram. A complete small design gives the interviewer something concrete to challenge. Deepen the risky edge after the end-to-end path is clear.

Each drawing should answer one question

An architecture view answers “who owns state and who communicates?” For a personal reading list, it might show browser, API, and a database, with a trust boundary at the API and authoritative records in the database. An arrow should name the operation or payload; a line alone leaves direction and meaning ambiguous.

A sequence view answers “in what order does one operation happen, and when is success returned?” Draw the client submitting a save, the application validating ownership, the database accepting the write, and the response. A failure timeline answers “which steps happened before the interruption, and what can each participant know afterward?” These views describe the same names and guarantees at different levels.

The C4 reference separates levels of structural detail and also offers dynamic and deployment views. You need only the views that answer the current question. Do not turn a timed interview into a notation exam. Consistent labels and honest boundaries matter more than elaborate icons.

Make synchronous requests and asynchronous messages visually distinct and say what an acknowledgment means. At this point you can label an asynchronous idea as future work without inventing delivery guarantees. Never use an animation to imply that data is durable before the mechanism has established that fact.

A worked decision record

Prompt: a small faculty team saves links privately and shares selected folders with colleagues. Initial assumptions: 200 active users/day, 20 actions each, a tenfold peak, one institution, and no offline editing. Average demand is about 0.046/s, peak about 0.46/s. This estimate suggests that basic correctness and operator effort are more important than introducing a distributed partitioning scheme. It does not prove any particular server's capacity.

Decision D1: use one application and one persistent database for links, folders, and membership. Reason: the important operation is deciding whether the requester may see a folder and then retrieving its entries. Co-locating authoritative records supports a straightforward reasoning path. Alternative: several independently deployed services would add communication and failure boundaries without a stated scaling or team requirement. Risk: a machine outage stops access. Revisit trigger: the measured workload or recovery objective no longer fits the current deployment.

Trace: request folder 17 as user 9; application establishes user identity; it checks membership against authoritative data; it reads the allowed links; it returns only those entries. A client-supplied folder ID is a lookup parameter, not evidence of permission. A useful test is that user 10 cannot read folder 17 merely by guessing its ID.

Changed requirement: ten departments now need isolated administration. Adapt the ownership model by making department scope explicit and checking it consistently. Do not immediately multiply every component by ten. First decide whether shared storage with enforced isolation satisfies the threat model and operating constraints. Later modules develop the implementation details.

Explain the reason before naming the component

Use a compact reasoning sentence: “Because confirmed membership changes must govern private reads, I will check authoritative access state before returning entries; this adds a read cost that I will measure.” Compare it with “I will use a cache because caches are fast.” The first establishes a guarantee and a cost; the second skips the problem and the stale-access risk.

When challenged, distinguish disagreement about requirements from disagreement about mechanisms. If the requirement changes from private notes to public links, some isolation decisions may relax. If the requirement stays private but a proposed cache can serve revoked access, the mechanism needs repair. Adapting to evidence is a design skill, not a loss of confidence.

Summarize at the end using scope, main state ownership, two key decisions, the most consequential failure, and a validation plan. Name uncertainty specifically: “I have not measured the large-folder read distribution, so pagination and capacity are the next test” is useful. “It should scale” is not.

Common mistakes are drawing everything at once, treating a component name as justification, answering a challenge by adding unexplained boxes, and using the full time without a conclusion. Optional rehearsal: record a five-minute explanation, then mark every sentence that lacks either a requirement, mechanism, consequence, or question. Editing the recording is additional practice, outside this session.

SYSTEM VIEW

A design conversation is an iterative loop

The loop explains how later evidence can revise an earlier decision. It is a reasoning map, not a network architecture.

Step through the mechanism, one transition at a time.

BOUNDED EXERCISE · 10 MINUTES

Spend ten minutes preparing a five-minute explanation of the faculty reading-list service. Write a six-line decision record and draw one architecture view plus one request sequence. Introduce a requirement that folder access revocation must apply to the next read and identify which decision needs more scrutiny.

State your assumptions. Show your reasoning.

1. Which view best exposes success before persistence?

- A. A request sequence
 - B. A list of product names
 - C. A server-cost table
-
-
-

2. What should a deep dive prioritize?

- A. The technology you know most buzzwords about
 - B. The decision with the greatest consequence for the scoped guarantees
 - C. Every component equally
-
-
-

3. In the supplied 45-minute framework, scope 5 + estimates 4 + APIs 6 + architecture 7 + deep dive 12 + failures 7 leaves how many minutes for synthesis?

4. What belongs in a decision record besides the chosen approach?

5. A changed requirement invalidates your initial design. How should you respond?

MENTAL MODEL

Use the clock to preserve a complete argument, then spend depth where a wrong assumption would hurt most.

References

Amazon Jobs: SDE II Interview Prep — <https://www.amazon.jobs/content/en/how-we-hire/sde-ii-interview-prep>

C4 model: Diagrams — <https://c4model.com/diagrams>

[C] Design a small single-server service without overengineering

drafted

LEARNING OBJECTIVES

Produce a coherent simple architecture from requirements.

Connect demand, retained bytes, and concurrency with capacity evidence.

Adapt the design to failure and changed workload.

RETRIEVE BEFORE READING

Assessment session

This catalog slot uses interview_002, a partial-design assessment. Start it in the Interview Center. The scheduled session is 60 minutes: 45 minutes of answering followed by 15 minutes of evidence-based feedback. The candidate brief is revealed when the attempt begins. Coaching mode permits requested hints; exam mode records hints and keeps the reference solution for the debrief. Do not use a normal lesson quiz as a substitute for this session.

BOUNDED EXERCISE · 45 MINUTES

Start the mapped interview, save your actual reasoning, then compare it with the rubric after submission.

State your assumptions. Show your reasoning.

MENTAL MODEL

Demonstration requires an attempted explanation and recorded evidence; opening this page awards no proficiency.

References

Amazon Jobs: SDE II Interview Prep — <https://www.amazon.jobs/content/en/how-we-hire/sde-ii-interview-prep>